

Understanding the attack vectors specific to Claude Code

### The 4 main vectors

Claude Code is an agent with access to the filesystem, Bash commands, and the internet. Its attack surface is broader than a simple text assistant.

**Vector 1: Prompt injection via external content.** When Claude reads an email, a GitHub issue, or an analyzed file containing malicious instructions, it may execute them. The attack exploits the confusion between data and instructions.

**Vector 2: Secret exfiltration via Bash.** If permissions are too broad, a malicious prompt can ask Claude to read `.env`, SSH keys, or locally stored tokens, then exfiltrate via an HTTP request.

**Vector 3: Supply chain via malicious skills and plugins.** A Snyk study (ToxicSkills, 2026) on 3,984 skills found 36.8% of skills with security flaws, including 534 with critical risk. The “rug pull” is the most insidious variant: a legitimate MCP that becomes malicious after being approved.

**Vector 4: Privilege escalation in multi-agent pipelines.** In an orchestration, a compromised agent can pass malicious instructions to subsequent agents. The outputs of one agent become the inputs of the next without intermediate validation.

### Quick risk matrix

| Scenario                 | Risk     | Immediate action                                      |
|--------------------------|----------|-------------------------------------------------------|
| Solo dev, public repos   | Medium   | Install an output-scanner hook                        |
| Team, sensitive codebase | High     | MCP vetting + injection hooks                         |
| Enterprise, production   | Critical | Least privilege, isolation and integrity verification |

### CVEs to know (selection 2025-2026)

| CVE                  | Severity       | Summary                                                          |
|----------------------|----------------|------------------------------------------------------------------|
| CVE-2025-53109/53110 | High           | Sandbox escape filesystem MCP                                    |
| CVE-2025-54135       | High           | RCE via prompt injection in Cursor                               |
| ADVISORY-CC-2026-001 | High           | Sandbox bypass, patch v2.1.34+                                   |
| ADVISORY-CC-2026-002 | Medium         | Deny rules silently dropped past 50 sub-commands, patch v2.1.90+ |
| CVE-2026-0755        | Critical (9.8) | RCE in gemini-mcp-tool (no patch)                                |

**Immediate action if you are on v2.1.33 or earlier:** update to v2.1.34+ to fix the sandbox bypass.

### Defense in depth

The principle is simple: each layer reduces the probability that an attack succeeds.

```
Minimal permissions (settings.json)
+ Tool whitelist per task
+ Reading MCPs before installation
+ JSONL logs for post-incident detection
+ Sandbox / ephemeral environment
```

**Basic permissions rule:** grant only what the task requires. For a read/analysis task, restrict the agent with

```
tools: Read, Grep, Glob or CLI --tools "Read,Grep,Glob" . An
```

allow rule only pre-approves a tool; it does not remove other tools.

### Audit with JSONL logs

Claude Code writes all its tool calls to the session JSONL file, `~/.claude/projects/{encoded-path}/{session-id}.jsonl`. These logs allow detecting abnormal behavior after the fact: unexpected reads of sensitive files, unplanned network calls, modification attempts outside scope.

```
# Inspect tool calls from a session
cat ~/.claude/projects/*/*.jsonl | jq
'.message.content[]?.name // empty'
```